Python Mastery

Level 1: Python Basics (Weeks 1-3)

These are topics you must know *before* writing any Django code. Mastery means you can write small scripts (50-100 lines) without referencing documentation constantly.

Category	Specific Topics
Syntax & Data Types	Variables, strings, integers, floats, booleans, None, type conversion, input/output
Data Structures	Lists (indexing, slicing, append, pop, comprehension), tuples, dictionaries (keys, values, items, get method), sets (union, intersection)
Control Flow	if/elif/else, for loops (range, enumerate, iterating dictionaries), while loops, break, continue, pass
Functions	Defining functions, return values, parameters vs arguments, default parameters, *args and **kwargs, scope (local vs global), lambda functions
Error Handling	try/except/else/finally, raising exceptions, common built-in exceptions (KeyError, ValueError, TypeError)
File I/O	Reading/writing text files (open, read, write, close, with statement), CSV handling using csv module

Verification Project: Build a command-line contact book that stores names and phone numbers in a dictionary, saves to a CSV file on exit, and loads on startup.

Level 2: Python Intermediate (Weeks 4-7)

These topics bridge the gap between scripting and professional application development. You need these *before* learning Django models or DRF serializers.

Category	Specific Topics
Modules & Packages	import vs from, creating your own modules, `__name__ == "__main__"`, understanding PYTHONPATH, pip install and requirements.txt
List & Dict Comprehensions	Nested comprehensions, conditional comprehensions, performance considerations
Built-in Functions	map, filter, zip, sorted, any, all, enumerate, reversed, min, max, sum, len, type, isinstance
Working with Dates/Times	datetime module (date, time, datetime, timedelta), strftime and strptime formatting, timezone awareness basics
Regular Expressions	re module basics (search, match, findall, sub), common patterns (email, phone, date), greedy vs non-greedy
Virtual Environments	venv module, creating and activating environments, why isolation matters, pip freeze
Debugging	print debugging, pdb basics (set_trace, step, continue, breakpoints), logging module (basicConfig, levels, handlers)
Working with JSON & APIs	json module (loads, dumps, load, dump), making HTTP requests using requests library (get, post, status codes, headers)

Verification Project: Build a weather CLI tool that takes a city name, fetches data from a public API (e.g., OpenWeatherMap), parses JSON, and displays formatted output with error handling for invalid cities or network failures.

Level 3: Python Advanced (Weeks 8-12)

These topics separate junior developers from mid-level engineers. You will use them constantly in Django, DRF, and your AI projects.

Category	Specific Topics
Object-Oriented Programming (OOP)	Classes and instances, `__init__` and `self`, instance methods vs class methods vs static methods, inheritance (single and multiple), method overriding, `super()`, encapsulation (public, protected, private conventions), properties (`@property` and setter), abstract base classes (ABC, abstractmethod), dataclasses
Decorators	Function decorators (timing, logging, access control), decorators with arguments, `@staticmethod`, `@classmethod`, `@property`, building custom decorators, common built-in decorators (`@lru_cache`, `@dataclass`)
Iterators & Generators	Iterator protocol (`__iter__`, `__next__`), `yield` vs `return`, generator expressions, memory efficiency benefits, `itertools` module (chain, cycle, product, combinations)
Context Managers	`with` statement for files and locks, building custom context managers using `__enter__` and `__exit__`, using `contextlib.contextmanager` decorator
Advanced Data Structures	`collections` module: deque (efficient appends/pops), defaultdict (auto-initialize), Counter (frequency counting), OrderedDict (insertion order), namedtuple (lightweight immutable objects)
Functional Programming	`functools` module: partial (freezing arguments), reduce, lru_cache (memoization), wraps (preserving metadata in decorators)
Concurrency Basics	Threading vs multiprocessing vs asyncio (high-level understanding), `time.sleep` and blocking, when to use each (I/O-bound vs CPU-bound), basics of `concurrent.futures` (ThreadPoolExecutor, ProcessPoolExecutor)

Category	Specific Topics
Testing	unittest framework (TestCase, setUp, tearDown, assertions), pytest basics (fixtures, parametrize, assert rewriting), mocking with unittest.mock (patch, Mock, MagicMock)
Type Hints	Basic type hints (int, str, list, dict, Optional, Union), type hints for functions and classes, using mypy for static checking
Packaging	`setup.py` or `pyproject.toml`, building distributable packages (`pip install -e .`), publishing to PyPI (basic understanding)

Verification Project: Build a small library management system with classes for Book, Member, and Library. Include decorators for logging method calls, a generator for iterating through overdue books, type hints throughout, and unit tests covering at least 80% of functionality. Use a JSON file for persistent storage.

Git Mastery

Level 1: Git Basics (Week 1 of Git Learning)

These commands handle 80% of your daily Git work. Master these before touching any collaborative project.

Command	What It Does	Most Common Use Case
`git init`	Creates a new Git repository in current folder	Starting a new project from scratch
`git clone <url>`	Downloads an existing repository to your machine	Getting a project from GitHub to work on locally
`git status`	Shows which files are changed, staged, or untracked	Checking what you have modified before committing
`git add <file>`	Stages a specific file for commit	Preparing a single file to be saved
`git add .`	Stages all changed and new files in current directory	Quick staging of everything before commit
`git add -p`	Stages parts of a file interactively (hunks)	Committing only specific changes within one file
`git commit -m "message"`	Saves staged changes with a descriptive message	Creating a save point after finishing a feature

`git commit -am "message"`	Stages all tracked files and commits in one command	Quick commits for small changes (skip git add)
`git log`	Shows commit history with hashes, authors, dates	Reviewing what changed and when
`git log --oneline`	Shows compact commit history (one line per commit)	Quick visual of recent changes
`git diff`	Shows unstaged changes (what you haven't added yet)	Reviewing modifications before staging
`git diff --staged`	Shows staged changes (what will go into next commit)	Double-checking what you are about to commit
`git rm <file>`	Removes file from both Git and working directory	Deleting files properly (not just via file explorer)
`git mv <old> <new>`	Renames or moves a file and stages the change	Renaming files without breaking history
`.gitignore` (file)	Lists patterns for files Git should never track (secrets, cache, dependencies)	Preventing `__pycache__/`, `.env`, `node_modules/` from being committed

Verification Task: Initialize a Git repo for your Python basics project, add all `.py` files, commit with a meaningful message, check status, and view the log.

Level 2: Git Intermediate (Week 2 of Git Learning)

These commands he working in a team or contributing to open source.

Command	What It Does	Most Common Use Case
`git branch`	Lists all local branches (current branch marked with *)	Seeing what branches exist on your machine
`git branch <name>`	Creates a new branch at current commit	Starting work on a new feature without affecting main
`git checkout <branch>`	Switches to an existing branch	Moving between features or fixes
`git checkout -b <branch>`	Creates and switches to a new branch in one command	Quick branch creation for a new feature
`git switch <branch>`	Newer alternative to checkout for switching branches	More intuitive branch switching (Git 2.23+)
`git switch -c <branch>`	Creates and switches to a new branch (newer syntax)	Same as `checkout -b` but clearer
`git merge <branch>`	Integrates changes from another branch into current branch	Bringing a completed feature back into main
`git branch -d <branch>`	Deletes a branch that has been merged	Cleaning up after merging a feature
`git branch -D <branch>`	Forces deletion of an unmerged branch	Removing abandoned or experimental branches
`git stash`	Temporarily saves uncommitted changes and cleans working directory	Switching branches without committing half-finished work

Command	What It Does	Most Common Use Case
`git stash pop`	Restores the most recent stash and removes it from stash list	Getting your saved work back
`git stash list`	Shows all stashed items	Remembering what you stashed earlier
`git stash drop`	Deletes a specific stash without applying it	Removing an obsolete stash
`git reset HEAD <file>`	Unstages a file (keeps changes in working directory)	Undoing `git add` before commit
`git reset --soft HEAD~1`	Undoes last commit but keeps changes staged	Fixing a commit message or adding forgotten files
`git reset --mixed HEAD~1`	Undoes last commit and unstages changes (default behavior)	Rewriting the last commit completely
`git reset --hard HEAD~1`	Undoes last commit AND discards all changes (dangerous)	Completely erasing a bad commit
`git revert <commit-hash>`	Creates a new commit that undoes a specific previous commit	Safely undoing a commit that has already been pushed
`git remote -v`	Lists all remote repositories and their URLs	Checking where your code will push to
`git remote add origin <url>`	Connects local repo to a remote on GitHub/GitLab	First-time setup after `git init`
`git push -u origin <branch>`	Pushes branch to remote and sets upstream tracking	First push of a new branch

Command	What It Does	Most Common Use Case
`git push`	Pushes current branch to its tracked remote	Sending commits to GitHub after upstream is set
`git pull`	Fetches and merges changes from remote to local branch	Getting teammates' latest work
`git fetch`	Downloads remote changes without merging	Seeing what changed before deciding to merge
`git clone --depth 1 <url>`	Shallow clone (only latest commit, no history)	Saving time and bandwidth for large repos

Verification Task: Create a new branch called `feature-logging`, add a logging module to your Python advanced project, commit it, merge back to main, delete the feature branch, and push everything to GitHub.

Level 3: Git Advanced (Week 3-4 of Git Learning)

These commands handle complex history rewriting, debugging, and team workflows. You need these when contributing to large projects or fixing messy commit histories.

Command	What It Does	Most Common Use Case
`git rebase <branch>`	Reapplies commits from current branch on top of another branch	Keeping a linear history instead of merge commits
`git rebase -i HEAD~n`	Interactive rebase for last n commits	Squashing, reordering, editing, or dropping commits
`git cherry-pick <commit-hash>`	Applies a specific commit from another branch to current branch	Taking one bug fix without merging entire branch

Command	What It Does	Most Common Use Case
`git bisect start`	Begins binary search to find which commit introduced a bug	Finding the exact commit that broke something
`git bisect bad`	Marks current commit as broken	Telling bisect where the problem exists
`git bisect good <commit>`	Marks a known-working commit	Setting the other end of the search range
`git bisect reset`	Ends bisect session and returns to original branch	Cleaning up after finding the bug
`git reflog`	Shows history of where HEAD has pointed (including lost commits)	Recovering commits lost by hard reset or rebase
`git reflog expire --expire=now --all`	Clears reflog (use with caution)	Removing sensitive data from Git history
`git filter-branch`	Rewrites history across many commits (powerful but complex)	Removing a large file accidentally committed everywhere
`git filter-repo`	Modern replacement for filter-branch (requires separate install)	Same as above but safer and faster
`git worktree add <path> <branch>`	Creates a separate working directory linked to same repo	Working on two branches simultaneously without stashing
`git worktree list`	Shows all worktrees attached to this repository	Tracking where your linked worktrees live
`git worktree remove <path>`	Deletes a worktree	Cleaning up when done with parallel work

Command	What It Does	Most Common Use Case
`git submodule add <url>`	Adds an external repository as a submodule	Including a library repo inside your main project
`git submodule update --init --recursive`	Clones and updates all nested submodules	Setting up a project after cloning it
`git blame <file>`	Shows who last modified each line of a file	Finding why a line of code was changed and by whom
`git grep "<pattern>"`	Searches tracked files for a pattern	Finding all occurrences of a function call across the codebase
`git shortlog -sn`	Shows commits grouped by author sorted by count	Seeing who contributes most to a project
`git cherry -v`	Shows commits in current branch not in upstream	Checking what commits haven't been pushed yet
`git push --force-with-lease`	Safer force push (fails if remote has unexpected changes)	Updating remote after interactive rebase
`git push --force`	Overwrites remote branch without safety checks (dangerous)	Only use on personal branches or when `--force-with-lease` fails
`git config --global alias.<name> '<command>'`	Creates a shortcut for a complex Git command	Making `git tree` show a beautiful graph log
`git gc`	Garbage collection—cleans up unnecessary files and optimizes	Reducing repository size after many changes

Command	What It Does	Most Common Use Case
`git maintenance start`	Enables automatic background repository optimization	Keeping large repos performant over time

Verification Task: Create a messy commit history intentionally (5 small commits), use interactive rebase to squash them into 2 meaningful commits, use `git reflog` to find a commit you thought was lost, then configure a `git tree` alias that shows `--oneline --graph --all --decorate`.

Django & Django REST Framework Mastery

Level 1: Django Basics (Weeks 1-3 of Django Learning)

These topics get you building traditional server-rendered web applications before adding REST APIs. Master these before touching DRF—they form the foundation that DRF extends .

Category	Specific Topics
Project Setup	Installing Django, `django-admin startproject`, `python manage.py runserver`, project vs apps structure, `manage.py` commands (startapp, migrate, createsuperuser, shell)
URLs & Routing	`urlpatterns` list, `path()` and `re_path()` functions, including app URLs using `include()`, named URL patterns and `reverse()`, namespace for reusable apps
Models & ORM	Defining models with fields (CharField, IntegerField, ForeignKey, ManyToManyField), `Meta` class options, `__str__` method, making migrations (`makemigrations`, `migrate`), querying with `Model.objects` (filter, exclude, get, create, update)
Views (Function-Based)	`request` object (GET, POST, user, session), returning `HttpResponse`, `render()` for templates, `redirect()`, `get_object_or_404()`, handling forms in views

Category	Specific Topics
Templates	Django Template Language (variables `{{ }}`, tags `{% %}`, filters `{{ value|filter }}`), template inheritance (`extends`, `block`), built-in tags (`for`, `if`, `url`, `csrf_token`), static files handling
Admin Interface	Registering models in `admin.py`, customizing list_display, list_filter, search_fields, creating superuser, admin actions
Forms	`forms.Form` vs `forms.ModelForm`, rendering forms in templates, CSRF protection, form validation (`clean_<field_name>`), handling form submission

Verification Project: Build a blog platform with user authentication using Django's built-in `User` model . Users can register, log in, create/edit/delete their own posts, and view all posts on the homepage. Use Django's admin interface to manage posts and users. Deploy it locally with SQLite (you will upgrade to PostgreSQL later).

Level 2: Django Intermediate (Weeks 4-6 of Django Learning)

These topics separate a beginner from a capable Django developer. You will need these for production applications and before moving to DRF .

Category	Specific Topics
Class-Based Views (CBVs)	`ListView` (displaying model lists), `DetailView` (single object), `CreateView`, `UpdateView`, `DeleteView`, overriding methods (`get_queryset`, `get_context_data`, `form_valid`), mixing `LoginRequiredMixin` with CBVs
Authentication & Authorization	`@login_required` decorator, `LoginRequiredMixin`, `UserCreationForm` for registration, login/logout views, password reset flow, permissions (`@permission_required`, checking `user.has_perm`), groups

Category	Specific Topics
Advanced ORM	`select_related()` and `prefetch_related()` for performance, aggregations (`Count`, `Sum`, `Avg`), annotating querysets, `Q` objects for complex lookups, `F` expressions for field-to-field comparisons, transactions (`atomic` blocks)
Middleware	Understanding request/response flow, built-in middleware (Authentication, Session, CSRF, Security), writing custom middleware, `process_request` and `process_response` hooks
Sessions & Cookies	Session framework (database vs cache-backed), storing/retrieving session data, setting expiration, cookie attributes (secure, httponly, samesite)
Static & Media Files	`STATIC_URL`, `STATICFILES_DIRS`, `collectstatic` command, serving user-uploaded files (`MEDIA_URL`, `MEDIA_ROOT`), using `ImageField` with Pillow
Messages Framework	`messages.add_message()` with different levels (success, error, warning, info), displaying messages in templates
Pagination	`Paginator` class, page numbers, `has_previous`/`has_next` template logic, integrating with `ListView` via `paginate_by`
Email Sending	`send_mail()` function, configuring email backend (console backend for development), SMTP settings for production
Testing Basics	`TestCase` class, `setUp()` and `setUpTestData()`, testing models (string representation, field validation), testing views (client.get/post, status codes, template used), test database isolation

Verification Project: Extend your blog platform—add comments with nested replies using self-referential ForeignKey, implement user profiles using `OneToOneField` with signal handlers to auto-create on registration, add pagination for posts (10 per page), write unit tests covering models and critical views, and send email notifications to authors when someone comments on their post.

Level 3: Django Advanced (Weeks 7-9 of Django Learning)

These topics are for production-grade, scalable applications. Some bridge directly into DRF concepts .

Category	Specific Topics
Custom Model Fields	Creating fields that store data differently than standard fields (e.g., encrypted fields, compressed fields)
Custom QuerySets & Managers	Subclassing `QuerySet` for reusable filters, `Manager.as_manager()` from queryset, chaining custom methods
Signals	Built-in signals (`pre_save`, `post_save`, `pre_delete`, `m2m_changed`), writing custom signals, signal receivers decorator, avoiding circular imports, signal dispatch timing
Caching	Per-view caching (`@cache_page`), template fragment caching, low-level cache API (`cache.set()`, `cache.get()`), cache backends (Memcached, Redis, database)
Logging	`LOGGING` dict in settings, loggers, handlers (console, file), formatters, log levels, integrating `django.utils.log`
Django Debug Toolbar	Installing, configuration, panels for SQL queries, cache, signals, headers, profiling, using in development
Security Deep Dive	SQL injection prevention (parameterized queries), XSS protection (escaping, `mark_safe` pitfalls), CSRF implementation, clickjacking protection (`X-Frame-Options`), clickjacking protection (`X-Frame-Options`), SSL/HTTPS settings (`SECURE_SSL_REDIRECT`, `HSTS`), setting `SECURE_BROWSER_XSS_FILTER` and `SECURE_CONTENT_TYPE_NOSNIFF`
Asynchronous Views (Django 3.1+)	`async def` views, `sync_to_async` for ORM calls, running async tasks, limitations compared to Channels

Category	Specific Topics
Custom Management Commands	Creating `management/commands/` directory, subclassing `BaseCommand`, adding arguments, using `handle()` method, cron integration via `python manage.py`
Internationalization (i18n)	`ugettext_lazy` for translatable strings, `{% trans %}`, `{% blocktranslate %}`, language detection middleware, creating `.po` files with `makemessages`
Third-Party Packages	`django-allauth` (social authentication), `django-crispy-forms` (styled forms), `django-filter` (queryset filtering), `django-import-export` (CSV/Excel), `celery` + `django-celery-beat` (background tasks), `django-extensions` (shell_plus, graph_models)

Verification Project: Add caching to your blog's homepage with Redis backend, implement logging to track 404 errors and slow queries, add `django-debug-toolbar` for local profiling, write a custom management command to clean old unconfirmed user accounts, integrate `django-allauth` for Google/GitHub login, and set up Celery to send comment notification emails asynchronously.

Level 4: Django REST Framework (DRF) Basics (Weeks 10-12)

Transition to API development—this is where you build the backend that your React and React Native apps will consume.

Category	Specific Topics
Installing DRF	`pip install djangorestframework`, adding `'rest_framework'` to `INSTALLED_APPS`, choosing authentication and permission classes in `REST_FRAMEWORK` settings
Serializers	`serializers.Serializer` vs `serializers.ModelSerializer`, field types (CharField, IntegerField, PrimaryKeyRelatedField), `create()` and `update()` methods, `read_only_fields`, `extra_kwargs`, validating data with `validate()` and field-level `validate_<field_name>`

Category	Specific Topics
Function-Based Views (FBVs) with DRF	`@api_view` decorator, `Response` objects vs `JsonResponse`, handling `GET`, `POST`, `PUT`, `DELETE` requests, manually handling serializers, `status` module for HTTP codes
Class-Based Views (CBVs) in DRF	`APIView` class, `get()`, `post()`, `put()`, `delete()` methods, permissions classes inside views (`permission_classes = [IsAuthenticated]`), handling request.user
GenericAPIView & Mixins	`GenericAPIView` (queryset, serializer_class attributes), `ListModelMixin`, `CreateModelMixin`, `RetrieveModelMixin`, `UpdateModelMixin`, `DestroyModelMixin`, combining mixins
Concrete View Classes	`ListAPIView`, `CreateAPIView`, `RetrieveAPIView`, `UpdateAPIView`, `DestroyAPIView`, `ListCreateAPIView`, `RetrieveUpdateDestroyAPIView`—these save huge boilerplate
ViewSets & Routers	`ViewSet` class (list, create, retrieve, update, partial_update, destroy methods), `ModelViewSet` (full CRUD), `ReadOnlyModelViewSet`, registering with `DefaultRouter` for automatic URL generation
Permissions	`IsAuthenticated`, `IsAdminUser`, `AllowAny`, `IsAuthenticatedOrReadOnly`, `DjangoModelPermissions`, custom permission classes (override `has_permission`, `has_object_permission`)
Authentication Classes	`SessionAuthentication` (works with Django login), `BasicAuthentication`, `TokenAuthentication` (built-in token model), `JSONWebTokenAuthentication` (JWT—requires djangorestframework-simplejwt)

Verification Project: Convert your blog platform into a REST API. Create serializers for Post, Comment, and User models. Implement endpoints: `GET /api/posts/` (list with pagination), `POST /api/posts/` (create, auth required), `GET /api/posts/<id>/` (detail), `PUT/PATCH/DELETE /api/posts/<id>/` (owner-only or admin-only). Add `GET /api/comments/` filterable by post_id. Use `ModelViewSet` with custom permission classes. Implement token authentication and test with Postman .

Level 5: Django REST Framework Advanced (Weeks 13-16)

These topics make your APIs production-ready, scalable, and secure .

Category	Specific Topics
Authentication Deep Dive	djangorestframework-simplejwt for JWT (access + refresh tokens), rotating refresh tokens, blacklisting, customizing token claims, integrating with React's localStorage/sessionStorage, djoser or dj-rest-auth for registration/password reset endpoints
Advanced Permissions	SAFE_METHODS for read-only access, object-level permissions checking ownership, permissions using request.user and obj.author, combining multiple permission classes, permission_required attribute on ViewSets
Filtering	django-filter integration, filterset_fields, custom FilterSet classes with complex lookups, SearchFilter (using search_fields), OrderingFilter (using ordering_fields), filtering by URL parameters
Pagination	PageNumberPagination (page param), LimitOffsetPagination (limit/offset), CursorPagination (performance for large datasets), custom pagination classes, pagination in ViewSets
Throttling	AnonRateThrottle (for unauthenticated), UserRateThrottle (for authenticated), ScopedRateThrottle (per view/action), custom throttling based on user roles, setting default throttle classes and rates
Versioning	AcceptHeaderVersioning, URLPathVersioning (/v1/ prefix), NamespaceVersioning, HostName Versioning, setting DEFAULT_VERSION and ALLOWED_VERSIONS
Nested Serializers & Relationships	Serializing ForeignKey (PrimaryKeyRelatedField, StringRelatedField, HyperlinkedRelatedField, nested serializer with many=True), depth option for simple nesting, writing custom to_representation for complex serialization, HyperlinkedModelSerializer for HATEOAS APIs

Category	Specific Topics
Validators in DRF	`UniqueValidator`, `UniqueTogetherValidator`, custom `validate()` methods on serializers, field-level `validators` list, using Django's model validators with DRF
Schema Generation & Documentation	`drf-spectacular` or `drf-yasg` for OpenAPI/Swagger schema, auto-generated interactive docs (`/swagger/`, `/redoc/`), `@extend_schema` decorator for customizing descriptions, examples, and response codes
Testing DRF APIs	`APITestCase` or `APITransactionTestCase`, `APIClient` for simulated requests, `force_authenticate()` for auth testing, testing status codes and response data, `reverse()` for named URLs, testing permissions and throttling
Performance Optimization	`select_related` and `prefetch_related` in `.queryset` attribute, `defer()` and `only()` to reduce fields, `SerializerMethodField` for computed fields (but watch for N+1), caching with `@cache_page` on API views, response compression with `django.middleware.gzip.GZipMiddleware`

Verification Project: Upgrade your blog API to production grade. Add JWT authentication with simple jwt and include registration endpoints via dj-rest-auth. Implement filtering by title and category, search by content, ordering by date. Use cursor pagination for infinite scroll. Add throttling (100 requests/hour for unauthenticated, 1000/day for authenticated). Generate Swagger documentation using drf-spectacular. Write API tests covering CRUD operations and permission checks. Measure performance with Django Debug Toolbar and optimize with select_related and caching.

NumPy (3-5 days)

Array creation: np.array(), np.zeros(), np.ones(), np.arange(), np.linspace()

Array operations: Element-wise math (+, -, *, /), broadcasting rules, dot product (np.dot() or @)

Indexing & slicing: Accessing rows, columns, subsets, boolean indexing (arr[arr > 5])

Reshaping & stacking: reshape(), concatenate(), vstack(), hstack()

Key functions: np.mean(), np.std(), np.sum(), np.min(), np.max(), np.where()

Pandas (5-7 days)

Data structures: Series (1D) vs DataFrame (2D), creating from dictionaries or lists

Data loading: pd.read_csv(), pd.read_json(), df.to_csv()

Inspecting data: df.head(), df.info(), df.describe(), df.shape, df.columns

Selecting data: df['column'], df[['col1', 'col2']], df.iloc[] (position-based), df.loc[] (label-based), boolean filtering (df[df['age'] > 25])

Handling missing values: df.isnull(), df.dropna(), df.fillna()

Grouping & aggregation: df.groupby(), df.agg(), df.merge() (joins)

Basic ML with Scikit-learn (2 weeks)

Train-test split: train_test_split() with random_state and stratify parameters

Preprocessing: StandardScaler (normalization), LabelEncoder (categories to numbers), OneHotEncoder

Classification models: LogisticRegression (binary), RandomForestClassifier (robust default), metrics.accuracy_score, classification_report (precision, recall, F1)

Regression models: LinearRegression, mean_squared_error, r2_score

Clustering: KMeans (unsupervised grouping)

Model persistence: joblib.dump() and joblib.load() (saving trained models for your Django backend)

Pipelines: make_pipeline() to chain preprocessing + model

PyTorch (2-3 weeks)

Tensors: Creating tensors (torch.tensor(), torch.zeros(), torch.randn()), tensor operations (reshape, transpose, device movement with .to('cuda'))

Autograd: requires_grad=True, backward() for automatic differentiation (understanding gradients without deriving math)

Dataset & DataLoader: torch.utils.data.Dataset (custom dataset class), DataLoader (batching, shuffling)

Neural network layers: nn.Linear (dense layer), nn.ReLU, nn.Dropout, nn.Sequential

Training
loop: optim.Adam or SGD, nn.MSELoss or nn.CrossEntropyLoss, zero_grad() → loss.backward() → optimiz er.step()

Saving & loading: torch.save(model.state_dict()), model.load_state_dict()

Hugging Face (1 week)

Pipelines: pipeline("text-generation"), pipeline("sentiment-analysis"), pipeline("feature-extraction") for embeddings

Loading models: AutoModel.from_pretrained(), AutoTokenizer.from_pretrained() (e.g., "gpt2", "bert-base-uncased")

Tokenization: tokenizer(text, padding=True, truncation=True, return_tensors="pt")

Model outputs: Accessing logits, hidden states, generated text with model.generate()

Prompt Engineering (1 week)

Basic prompt structures: Zero-shot (direct question), few-shot (examples included), chain-of-thought (reasoning steps)

System vs user prompts: Role assignment ("You are a helpful assistant"), constraints ("Answer in 2 sentences")

Temperature & top-p: Controlling randomness (temperature=0 for deterministic, 0.7 for creative)

Prompt chaining: Breaking complex tasks into sequential API calls (e.g., summarize → extract keywords)

LangChain (2 weeks)

Core concepts: LLM (wrapping API calls), PromptTemplate (reusable prompts), OutputParser (structured responses)

Chains: LLMChain (prompt → LLM → parser), SimpleSequentialChain (chain multiple chains)

Memory: ConversationBufferMemory (chat history), ConversationSummaryMemory (compressed summaries)

Tools & Agents: Tool (wrapping Python functions), initialize_agent() with AgentType.ZERO_SHOT_REACT_DESCRIPTION (LLM decides which tool to call)

RAG (Retrieval-Augmented Generation) (2 weeks)

Document loading: DirectoryLoader, PyPDFLoader, TextLoader from LangChain

Chunking: RecursiveCharacterTextSplitter (chunk_size=1000, chunk_overlap=200)

Embeddings: HuggingFaceEmbeddings or OpenAI embeddings (converting text to vectors)

Vector stores: ChromaDB or FAISS (store and search embeddings locally)

Retrieval: vectorstore.similarity_search(query, k=4) → retrieve relevant chunks

RAG chain: Retrieve → combine chunks into context → send to LLM with question → generate grounded answer

Fine-tuning (1-2 weeks)

LoRA (Low-Rank Adaptation): peft.LoraConfig (rank r=8, alpha, dropout)—fine-tunes only 1% of parameters

Training arguments: transformers.TrainingArguments (learning_rate, per_device_batch_size, num_epochs)

Trainer API: Trainer(model, args, train_dataset, eval_dataset) (abstracts the training loop)

Dataset preparation: JSONL format (input → output pairs), tokenizing with labels

Saving & loading PEFT adapters: model.save_pretrained("lora_adapter"), PeftModel.from_pretrained(base_model, "lora_adapter")